

Insurance Claims to E-commerce

Spring Boot microservices domain conversion assessment

Executive summary

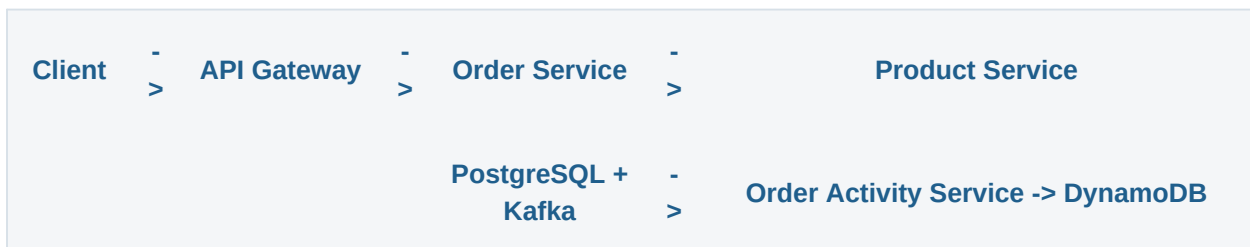
The repository contains six deployable Spring Boot applications: two domain services, one event-driven activity service, and three platform services. The most direct one-to-one conversion is policy-service to product-service, claims-service to order-service, and activity-service to order-activity-service. The gateway, configuration server, and discovery server retain their current roles.

Recommended conversion: Product Service + Order Service + Order Activity Service, while preserving every existing integration style and infrastructure component.

Inspection status

- Repository inspection was read-only.
- No source or configuration files were modified during the assessment.
- The working tree was clean at inspection time.
- Supporting PostgreSQL, Kafka, and DynamoDB Local containers are infrastructure, not separate microservices.

Proposed service flow



1. Current services and proposed responsibilities

claims-service	Creates, stores, retrieves, and lists claims. Validates a policy through OpenFeign and Eureka, writes to PostgreSQL, and publishes CLAIM_CREATED to Kafka.	order-service: creates and retrieves orders, validates the referenced product, persists orders, and publishes ORDER_CREATED.
policy-service	Creates, stores, retrieves, and lists policies in PostgreSQL, including coverage, vehicle, customer, date, and lifecycle data.	product-service or catalog-service: manages products, SKUs, prices, categories, sellers, and availability.
activity-service	Runs without a web server, consumes claim events from Kafka, and records activity history in DynamoDB.	order-activity-service: consumes order events and records order activity or audit history in DynamoDB.
api-gateway	Routes policy and claim API paths through Eureka-backed load balancing on port 8222.	Routes product and order API paths using the same gateway architecture.
config-server	Serves centralized native configuration from config-server/config-repo on port 8888.	Same responsibility; only service configuration names and routes change.
discovery-server	Provides the Eureka service registry on port 8761.	Same responsibility; no domain behavior changes.

The domain mapping preserves the existing service boundaries: policy-service is the referenced master-data service, claims-service is the transactional service that validates that reference, and activity-service is the asynchronous event consumer.

Current claim creation sequence

- The gateway forwards a request to claims-service.
- claims-service calls policy-service through OpenFeign and Eureka.
- The policy must exist and have ACTIVE status.
- The claim is saved to its PostgreSQL database with SUBMITTED status.
- A CLAIM_CREATED event is published to the claim-events Kafka topic.
- activity-service consumes the event and writes an activity record to DynamoDB.

Equivalent order creation sequence

- The gateway forwards a request to order-service.
- order-service calls product-service through OpenFeign and Eureka.
- The product must exist and be available.
- The order is saved to its PostgreSQL database with PLACED status.
- An ORDER_CREATED event is published to the order-events Kafka topic.
- order-activity-service consumes the event and writes an activity record to DynamoDB.

2. Architecture and technologies to preserve

The conversion requires domain renaming and field remapping, not an architectural redesign. All existing frameworks, data stores, protocols, and interaction patterns can remain in place.

- Java 17
- Spring Boot 4.1.1
- Spring Cloud 2025.1.3
- Spring Cloud Gateway WebFlux
- Eureka service discovery
- Spring Cloud Config Server with the native configuration repository
- OpenFeign and Spring Cloud LoadBalancer
- Spring MVC REST APIs
- Spring Data JPA and Hibernate
- Flyway database migrations
- PostgreSQL with one database per domain service
- Apache Kafka for asynchronous domain events
- AWS SDK for DynamoDB activity persistence
- Lombok, Jakarta Validation, Actuator, Maven, and Docker Compose

No payment, shipping, authentication, inventory, saga, event-sourcing, or additional microservice pattern is proposed. Those capabilities are outside the current architecture.

Platform service impact

- api-gateway keeps the same technology and port; only route IDs, service names, and paths change.
- config-server keeps the same native repository; service configuration filenames are renamed.
- discovery-server keeps the same Eureka configuration and port.
- Kafka and DynamoDB Local remain unchanged except for domain-specific topic, table, and container names.

3. claims-service to order-service

This service remains the central transactional service. It will create, store, retrieve, and list orders; synchronously validate a referenced product; and asynchronously publish an order-created event.

Class and API renames

Current	Proposed
ClaimsServiceApplication	OrderServiceApplication
ClaimController	OrderController
ClaimService	OrderService
Claim	Order
ClaimRepository	OrderRepository
ClaimMapper	OrderMapper
ClaimResponse	OrderResponse
CreateClaimRequest	CreateOrderRequest
ClaimStatus	OrderStatus
ClaimType	OrderType
ClaimCreatedEvent	OrderCreatedEvent
ClaimEventProducer	OrderEventProducer
PolicyClient / PolicyResponse	ProductClient / ProductResponse
/api/v1/claims	/api/v1/orders
claim-events / CLAIM_CREATED	order-events / ORDER_CREATED

Exception renames

- PolicyNotFoundException -> ProductNotFoundException
- PolicyInactiveException -> ProductUnavailableException
- PolicyServiceUnavailableException -> ProductServiceUnavailableException
- Update GlobalExceptionHandler methods, error labels, and messages accordingly.

4. Domain field and persistence mapping

Order service fields

Current	Proposed
policyId	productId
customerId	customerId
claimType	orderType
description	description or orderNotes
amountRequested	totalAmount
ClaimStatus.SUBMITTED	OrderStatus.PLACED
claims table	orders table
insurance_claims database	ecommerce_orders database

Files requiring updates

- All Java files under claims-service/src/main/java/com/insurance/claims_service
- claims-service/pom.xml
- claims-service/src/main/resources/application.properties
- claims-service/src/main/resources/db/migration/V1__create_claims_table.sql
- claims-service/docker-compose.yml
- ClaimsServiceApplicationTests and its package path

The current event record exists independently in the producer and consumer services. Both copies must be renamed and kept structurally identical.

5. policy-service to product-service

This service remains the independently persisted master-data service. It will expose create, list, and get-by-ID operations for products and provide product availability data to order-service.

Class renames

Current	Proposed
PolicyServiceApplication	ProductServiceApplication
PolicyController	ProductController
PolicyService	ProductService
Policy	Product
PolicyRepository	ProductRepository
PolicyMapper	ProductMapper
PolicyResponse	ProductResponse
CreatePolicyRequest	CreateProductRequest
PolicyStatus	ProductStatus
CoverageType	ProductCategory
PolicyNotFoundException	ProductNotFoundException
/api/v1/policies	/api/v1/products

Product field mapping

Current	Proposed
policyNumber	sku
customerId	sellerId
vehicleVin	productName
coverageType	category
coverageLimit	price
effectiveDate	availableFrom
expirationDate	availableUntil
policies table	products table
insurance_policy database	ecommerce_products database

6. activity-service and platform configuration

Activity service conversion

Current	Proposed
activity-service	order-activity-service
ActivityServiceApplication	OrderActivityServiceApplication
ClaimEventConsumer	OrderEventConsumer
ClaimCreatedEvent	OrderCreatedEvent
ClaimActivityRepository	OrderActivityRepository
consumeClaimCreated	consumeOrderCreated
claim-events	order-events
claim-activity DynamoDB table	order-activity DynamoDB table
claimId / policyId	orderId / productId

Configuration files

- config-server/config-repo/claims-service.yml -> order-service.yml
- config-server/config-repo/policy-service.yml -> product-service.yml
- config-server/config-repo/activity-service.yml -> order-activity-service.yml
- config-server/config-repo/api-gateway.yml: update route IDs, lb:// service names, and URL predicates
- Update Spring application names in each service application.properties or application.yml file
- Update the OpenFeign target from policy-service to product-service
- Update Kafka deserialization trusted package and default event type
- Update Docker Compose database, service, container, and volume names where they contain insurance terminology

Global package identity

- Change Java package root com.insurance to com.ecommerce across main and test sources.
- Change Maven groupId com.insurance to com.ecommerce in all six pom.xml files.
- ConfigServerApplication, DiscoveryServerApplication, and ApiGatewayApplication keep their functional class names.

7. Recommended implementation boundary

The conversion should be implemented as a coordinated rename across source code, package paths, service discovery names, gateway routes, database schemas, Kafka contracts, DynamoDB attributes, Docker resources, and context-load tests. Ports and runtime topology can remain unchanged.

In scope

- One-to-one service and class renaming
- E-commerce field names and enums
- REST routes and Feign contract changes
- PostgreSQL schema and database name changes
- Kafka topic and event contract changes
- DynamoDB table and attribute changes
- Configuration, Docker Compose, Maven coordinates, packages, and tests

Explicitly outside the current conversion

- Adding new microservices
- Introducing payments, inventory reservations, carts, shipping, or authentication
- Changing the database-per-service model
- Replacing REST, Kafka, PostgreSQL, DynamoDB, Eureka, or Config Server
- Introducing sagas, CQRS, event sourcing, or new deployment technologies

Approval gate: no conversion changes should be made until the proposed Product, Order, and Order Activity mapping is approved.

Prepared from a read-only inspection of the repository.